```sql
-- ============================================
-- UPDATE SAMPLE DATA
-- ============================================

-- 1. Add additional category for trousers
INSERT INTO Category (category_name)
VALUES
    ('Trousers');

-- 2. Add new product in category 'Trousers'
INSERT INTO Product (product_name, price, category_id)
VALUES
    (
        'Otto Trousers',
        1359.00,
        (SELECT category_id
         FROM Category
         WHERE category_name = 'Trousers')
    );

-- 3. Updated dress price for 'Emma Dress'
UPDATE Product
    SET price = 1500.00
    WHERE product_name = 'Emma Dress';

-- 4. Deleting an order item for customer order
DELETE FROM OrderItem
WHERE Order_id = 1 AND product_id = 1;
```

Statistics 1

Name	Value
Updated Rows	1
Execute time	0.004s
Start time	Tue Oct 13 00:34:15 BST 2026
Finish time	Tue Oct 13 00:34:15 BST 2026
Query	-- 1. Add additional category for trousers INSERT INTO Category (category_name) VALUES ('Trousers')

```sql
-- 2. Add new product in category 'Trousers'
INSERT INTO Product (product_name, price, category_id)
VALUES
    (
        'Otto Trousers',
        1359.00,
        (SELECT category_id
         FROM Category
         WHERE category_name = 'Trousers')
    );

-- 3. Updated dress price for 'Emma Dress'
UPDATE Product
    SET price = 1500.00
    WHERE product_name = 'Emma Dress';

-- 4. Deleting an order item for customer order
DELETE FROM OrderItem
WHERE Order_id = 1 AND product_id = 1;
```

Statistics 1

Name	Value
Updated Rows	1
Execute time	0.003s
Start time	Tue Oct 13 00:34:46 BST 2026
Finish time	Tue Oct 13 00:34:46 BST 2026
Query	INSERT INTO Product (product_name, price, category_id) VALUES ('Otto Trousers', 1359.00, (SELECT category_id FROM Category WHERE category_name = 'Trousers'))

```sql
-- 3. Updated dress price for 'Emma Dress'
UPDATE Product
    SET price = 1500.00
    WHERE product_name = 'Emma Dress';

-- 4. Deleting an order item for customer orde
DELETE FROM OrderItem
WHERE Order_id = 1 AND product_id = 1;

-- 5. Adding a different item for customer ord
INSERT INTO OrderItem (order_id, product_id, q
```

Statistics 1

Name	Value
Updated Rows	1
Execute time	0.002s
Start time	Tue Oct 13 00:35:20 BST 2026
Finish time	Tue Oct 13 00:35:20 BST 2026
Query	UPDATE Product SET price = 1500.00 WHERE product_name = 'Emma Dress'

```sql
-- 4. Deleting an order item for customer order
DELETE FROM OrderItem
WHERE Order_id = 1 AND product_id = 1;

-- 5. Adding a different item for customer order where order number is 1
INSERT INTO OrderItem (order_id, product_id, quantity)
```

Statistics 1

Name	Value
Updated Rows	0
Execute time	0.001s
Start time	Tue Oct 13 00:35:32 BST 2026
Finish time	Tue Oct 13 00:35:32 BST 2026
Query	DELETE FROM OrderItem
	WHERE Order_id = 1 AND product_id = 1

```sql
-- 2. Selecting full customer information and items ordered, ordering by order_date
SELECT
    C.customer_id,
    C.first_name,
    C.last_name,
    C.email,
    O.order_id,
    O.order_date,
    I.product_id,
    P.product_name
FROM customer AS C
JOIN CustomerOrder AS O ON C.customer_id = O.customer_id
JOIN OrderItem AS I ON O.order_id = I.order_id
JOIN Product AS P ON I.product_id = P.product_id
ORDER BY customer_id, order_id;

-- 3. Selecting full customer information with left joins to show all customers regardless of whether they have ordered an item
SELECT
```

stomer(+) 1 ×

ELECT C.customer_id, C.first_na | Enter a SQL expression to filter results (use Ctrl+Space)

	123 customer_id	A-Z first_name	A-Z last_name	A-Z email	123 order_id	order_date	123 product_id	A-Z product_name
	1	Anna	Hansen	anna@example.com	1	2026-09-15	2	Sofia Top
2	1	Anna	Hansen	anna@example.com	3	2026-09-17	3	Ingrid Skirt
3	2	Emma	Olsen	emma@example.com	2	2026-09-16	1	Aurora Dress

```sql
-- 3. Selecting full customer information with left joins to show all customers regardless of whether they have ordered an item
SELECT
    C.customer_id,
    C.first_name,
    C.last_name,
    C.email,
    O.order_id,
    O.order_date,
    I.product_id,
    P.product_name
FROM customer C
LEFT JOIN CustomerOrder O ON C.customer_id = O.customer_id
LEFT JOIN OrderItem I ON O.order_id = I.order_id
LEFT JOIN Product P ON I.product_id = P.product_id
ORDER BY customer_id, order_id;

-- 4. Calculating sales revenue for each category
```

customer(+) 1

SELECT C.customer_id, C.first_name, C.last_name | Enter a SQL expression to filter results (use Ctrl+Space)

Grid | Text

	customer_id	first_name	last_name	email	order_id	order_date	product_id	product_name
1	1	Anna	Hansen	anna@example.com	1	2026-09-15	2	Sofia Top
2	1	Anna	Hansen	anna@example.com	3	2026-09-17	3	Ingrid Skirt
3	2	Emma	Olsen	emma@example.com	2	2026-09-16	1	Aurora Dress
4	3	Sofie	Berg	sofie@example.com	[NULL]	[NULL]	[NULL]	[NULL]

```sql
-- 4. Calculating sales revenue for each category
SELECT
    C.category_name,
    SUM(P.price * OI.quantity) AS total_revenue
FROM Category C
JOIN Product P ON C.category_id = P.category_id
JOIN OrderItem OI ON P.product_id = OI.product_id
GROUP BY C.category_id, C.category_name
ORDER BY total_revenue DESC;

-- 5. Restricting sales revenue analysis to categories with
SELECT
    C.category_name,
    SUM(P.price * OI.quantity) AS total_revenue
```

ategory 1 ×

ELECT C.category_name, SUM(P.price * OI.quanti | Enter a SQL expressio

	A-Z category_name	123 total_revenue
1	Tops	2,580
2	Dresses	2,490
3	Skirts	1,790

```sql
-- 7. Query to find latest order date and order id from any given customer, showing customers with no orders as well
SELECT
    C.customer_id,
    C.first_name,
    C.last_name,
    MAX(order_date) AS latest_order_date,
    MAX(order_id) AS latest_order_id
FROM customer C
LEFT JOIN CustomerOrder CO ON C.customer_id = CO.Customer_id
GROUP BY C.customer_id
ORDER BY C.customer_id;

-- 8. Query for average value of orders for customer '1'
SELECT
```

customer 1

SELECT C.customer_id, C.first_name, C.last_name | Enter a SQL expression to filter results (use Ctrl+Space)

Grid | Text

	customer_id	first_name	last_name	latest_order_date	latest_order_id
1	1	Anna	Hansen	2026-09-17	3
2	2	Emma	Olsen	2026-09-16	2
3	3	Sofie	Berg	[NULL]	[NULL]

```sql
-- 9. Query for average value of orders for each customer
SELECT
    C.customer_id,
    C.first_name,
    C.last_name,
    AVG(order_total) AS average_order_value
FROM Customer C
JOIN (
        SELECT
            CO.order_id,
            CO.customer_id,
            SUM(P.price * OI.quantity) AS order_total
        FROM CustomerOrder CO
        JOIN OrderItem OI ON CO.order_id = OI.order_id
        JOIN Product P ON OI.product_id = P.product_id
        GROUP BY CO.order_id, CO.customer_id
    ) AS order_values ON c.customer_id = order_values.customer_id
GROUP BY C.customer_id, C.first_name, C.last_name
ORDER BY average_order_value DESC;
```

customer 1

SELECT C.customer_id, C.first_name, C.last_name

Enter a SQL expression to filter results (use Ctrl+Space)

	123 customer_id	A-Z first_name	A-Z last_name	123 average_order_value
1	2	Emma	Olsen	2,490
2	1	Anna	Hansen	2,185